

EMBEDDED SYSTEM WITH IOT EXPERIMENT NO. 2

HOME APPLIANCE AUTOMATION USING IOT

Name: *Raphael S. Nano*

Course/Section: *CPE162P / CI*

Date of Performance: *February 27-29, 2024*

Date of Submission: *February 29, 2024*

Ma'am Cyrel Manlises

Instructor

DISCUSSION

Appliance controllers provide a wide range of functionality, from timers to complicated IoT devices that may manage and optimize the efficiency of domestic appliances. Appliance controllers have become a vital part of modern houses, increasing convenience while also promoting cost-effectiveness and energy conservation. As a result, it is critical to highlight an appliance controller's capabilities and building methods in order to further expand its usefulness. With that said, a home appliance automation system based on IoT was built and duplicated utilizing the ESP8266 controller and the WiFi connector, with the servo motor, passive buzzer, and LEDs serving as the controlled devices.

For experiment 2, the objectives are as follows:

- 1. To demonstrate how a home appliance automation using IoT works.*

2. To be able apply what you have learned in logic circuits and switching theory, microprocessors, CPE160P, and CPE161P.

3. To be able to apply embedded system with IoT using Arduino kit

4. To be able to use ThingSpeak for the data collection and analysis.

The materials involved in constructing the Home Appliance Automation Using IoT are the ESP8266 WiFi module, LEDs, passive buzzer, servo motor, infrared remote, and infrared receiver.

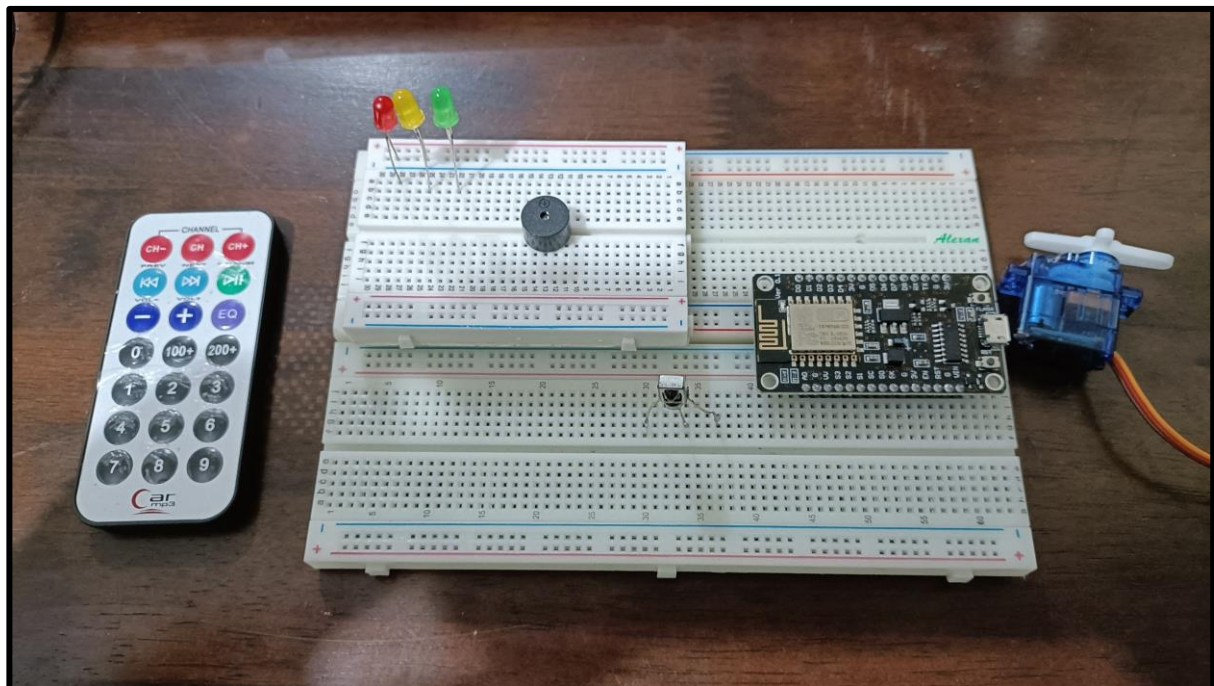


Figure 1. Major Components for the Experiment

For the external libraries, ESP8266WiFi.h and ThingSpeak.h are some of the separate libraries included. Secrets.h is a file with definitions for ESP8266WiFi.h. For the infrared remote and receiver, IRremoteESP8266.h, IRrecv.h, and IRutils.h are used. Servo.h was implemented for the servo motor.

```
#include <ESP8266WiFi.h>
#include "ThingSpeak.h"
#include <Arduino.h>

#include "IRremoteESP8266.h"
#include <IRrecv.h>
#include <IRutils.h>

#include "Servo.h"
```

Figure 2. External Libraries Included

The primary aim of experiment 2 is to build an electronic device that can be used to control all the home appliances using Bluetooth technology, wherein the system will be used to switch ON/OFF the devices by giving commands using the IR remote. Additionally, Thingspeak will

be used for the collection of the data and analysis.

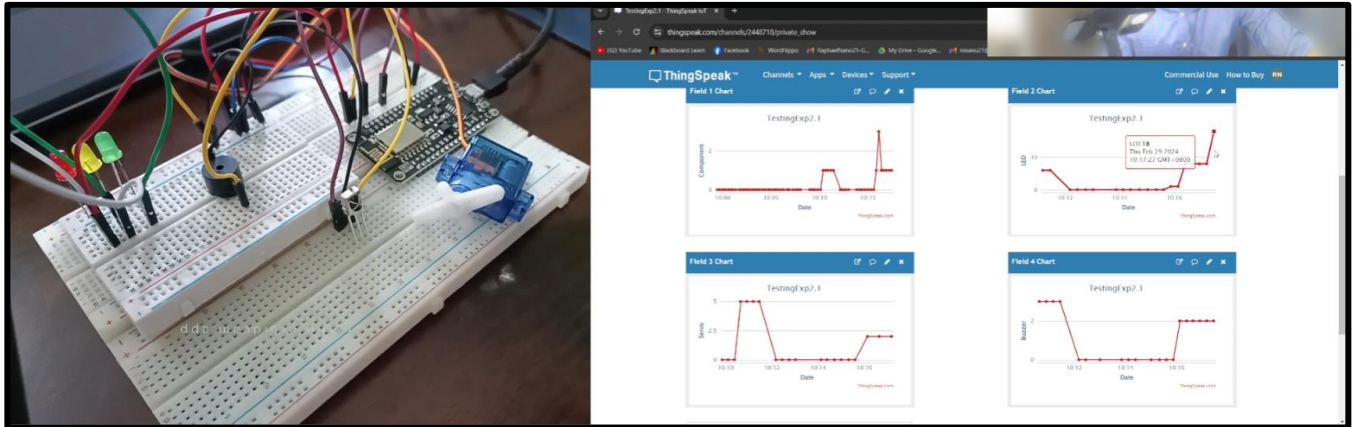


Figure 3. Sample Out put of the Experiment

The main component of this experiment is the IR remote and receiver, wherein the component to be controlled is decided by the value pressed in the remote. If 1 is pressed, the LEDcount will add 1, 2 for the servocount, and 3 for the buzzcount. In order to turn them all off, number 5 is pressed. The thingvalue definition is the value that is being out put in ThingSpeak, wherein each component is valued accordingly.

```

void remote() {
  if (irrecv.decode(&results)) {
    irdata = results.value;
    switch (irdata) {
      case 16724175:
        irval = 1;
        LEDflag = 1;
        thingvalue = 1;
        digitalWrite(16, HIGH);
        servo.write(0);
        break;
      case 16718055:
        irval = 2;
        servoflag = 1;
        thingvalue = 2;
        digitalWrite(16, LOW);
        servo.write(90);
        delay(500);
        servo.write(180);
        delay(500);
        break;

```

```

      case 16743045:
        irval = 3;
        buzzflag = 1;
        thingvalue = 3;
        playTone(2000, 1000);
        break;
      case 16726215:
        irval = 5;
        buzzflag = 0;
        LEDflag = 0;
        servoflag = 0;
        thingvalue = 0;
        digitalWrite(16, LOW);
        servo.write(0);
        break;
    }
    Serial.println(irdata);
    irrecv.resume();
  }
}

```

Figure 4. IR Remote Main Function

To connect the ESP8266 to Thingspeak, first establish a WiFi connection. The code below attempts to connect to a certain network via Wi-Fi (fh_dbe600) and makes multiple attempts until an adequate connection is established. It provides information to the user via the Serial Monitor, indicating the status of the connection attempt. After connecting to WiFi, the IR remote() function will be activated.

```

if(WiFi.status() != WL_CONNECTED){
  Serial.print("Attempting to connect to SSID: ");
  Serial.println("fh_dbe600");
  while(WiFi.status() != WL_CONNECTED){
    WiFi.begin(ssid, pass);
    Serial.print(".");
    delay(5000);
  }
  Serial.println("\nConnected.");
}

```

Figure 5. WiFi Function

Afterwards, the corresponding count for each component will be passed to the corresponding fields. Hence, all throughout the span of days, the counts of the usage of components will continue to increase.

```
ThingSpeak.setField(1,thingvalue);
ThingSpeak.setField(2,LEDcount);
ThingSpeak.setField(3,servocount);
ThingSpeak.setField(4,buzzcount);

int x = ThingSpeak.writeFields(myChannelNumber, myWriteAPIKey);
if (x == 200) {
    Serial.println("Channel update successful.");
} else {
    Serial.println("Problem updating channel. HTTP error code " + String(x));
}
```

Figure 6. Thingspeak Function

In the main loop, the WiFi connection will first be established, then remote() function is first called. Depending on what was pressed, the component will be activated: LEDs for 1, servo motor for 2, and buzzer for 3.

```
void loop() {
    if (WiFi.status() != WL_CONNECTED) {
        Serial.print("Attempting to connect to SSID: ");
        Serial.println("fh_dbe600");
        while (WiFi.status() != WL_CONNECTED) {
            WiFi.begin(ssid, pass);
            Serial.print(".");
            delay(5000);
        }
        Serial.println("\nConnected.");
    }

    remote();
    delay(300);
}
```

Figure 7. Main Loop Function

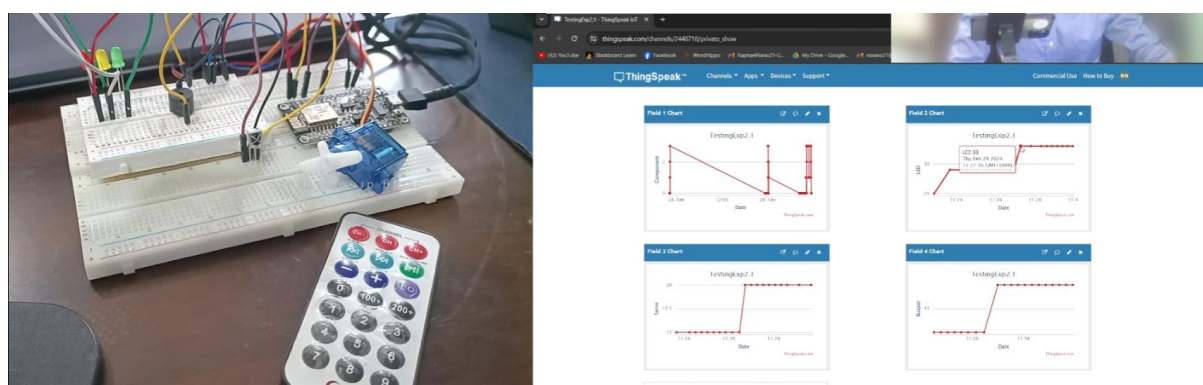
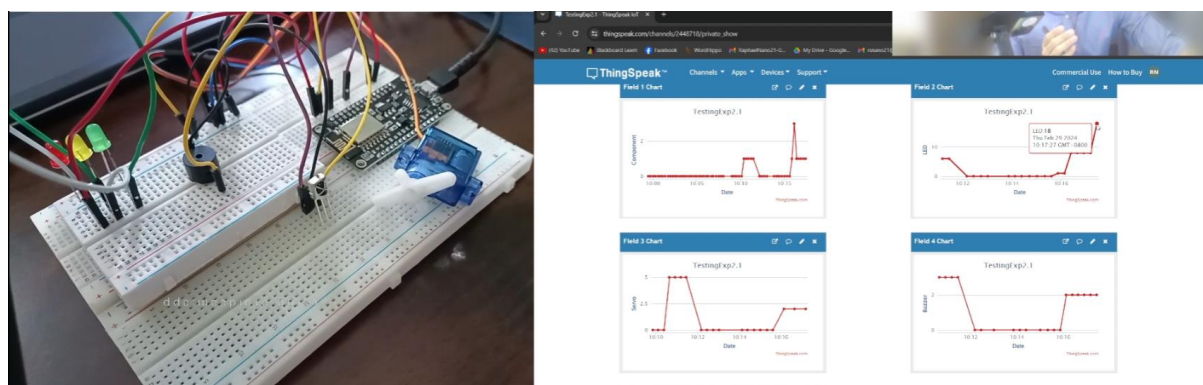
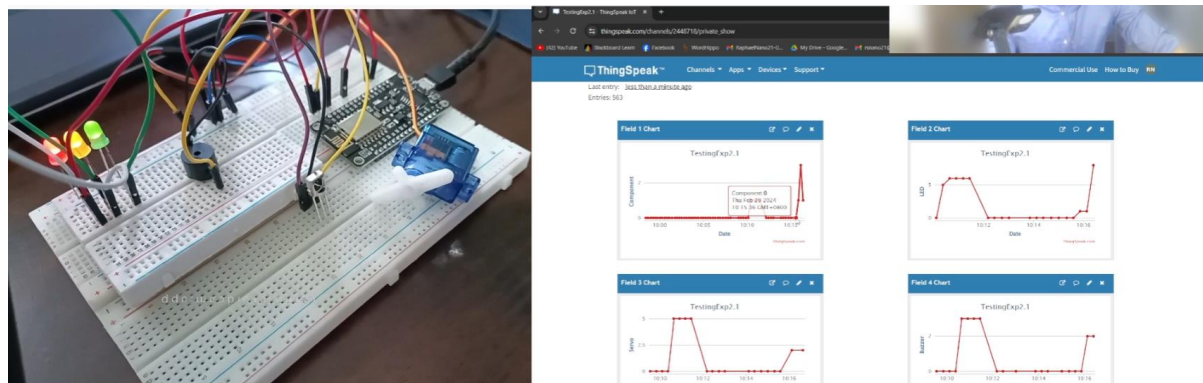


Figure 8. Final Outputs of Experiment 2

RESULTS

The Home Appliance Automation Using IoT heavily relies on the input of the user through the infrared remote. The input is then received by the IR receiver and passes the value to the ESP8266, the microcontroller. As per the outputs of the system and the ThingSpeak, the system shows the LEDs, servo motor, and buzzer being activated, and multiple fields were being successfully updated every 15 seconds. In the three-day span, the component that was ON the longest time and the most times is the LEDs. The appliance controller in a home frequently focuses on the prevalent light bulbs, which are required for everyday duties and ambiance. With the advancement of smart home technology, controlling these light bulbs has been smoothly linked into household administration, providing ease through remote access and customization features.

CONCLUSION

In conclusion, the Home Appliance Automation Using IoT works by using a switch case for the IR remote that determines what component will be activated and what value will be passed to ThingSpeak. Every press of the button from the IR remote would automatically update the system. The second goal was met by properly connecting the lighting up of LEDs, the rotation of the servo motor, and the alarming of the buzzer, depending on which button was pressed, and by synchronizing the component value in ThingSpeak, demonstrating the use of microprocessors, logic circuits, switching theory, CPE160P, and CPE161P lessons. The IoT portion was successfully integrated with the use of the communication and data writing between ESP8266 and Thingspeak, which was adequately performed as the graphs can prove the outputs of the component values. Hence, the Home Appliance Automation Using IoT was successfully

developed with its WiFi connection and the synchronization of outputs of the IR remote and output devices in Thingspeak.